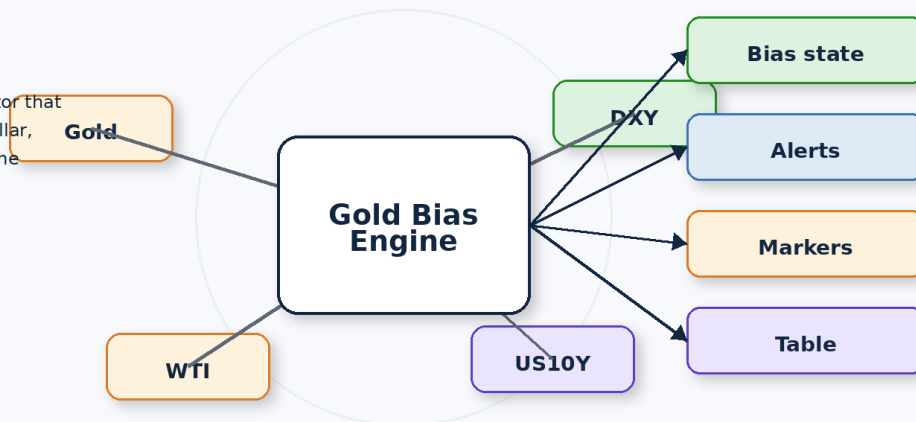


Macro Gold Bias Dashboard

Illustrated Documentation Pack

Tutorial + Whitepaper

A practical TradingView indicator that combines gold, oil, the U.S. dollar, and yields into a single real-time macro signal for gold.



Prepared for the user's Pine Script project • April 2026

Macro Gold Bias Dashboard — Design Whitepaper

A technical explanation of the model, architecture, scoring logic, and practical limitations

Prepared from the current Pine Script revision, including the typed oilScore fix.

Contents

- 1. Abstract
- 2. Problem statement and design goals
- 3. System architecture
- 4. Data layer and state classification
- 5. Scoring and bias mapping
- 6. Alert semantics and visualization
- 7. Design rationale
- 8. Limitations and failure modes
- 9. Validation framework
- 10. Extension roadmap
- Appendix A. Parameter reference
- Appendix B. Pseudocode

1. Abstract

The Macro Gold Bias Dashboard is a deterministic Pine Script indicator that translates four market series—gold, WTI oil, DXY, and a Treasury-yield proxy—into an interpretable bias state for gold. The model is intentionally transparent: every transformation is readable in code, and every output can be traced back to a compact set of rules.

The purpose of the model is not to forecast exact returns. Its purpose is to summarize whether the current macro mix is supportive, neutral, or hostile for gold, and to expose that view through chart coloring, markers, a dashboard table, and alert conditions.

This whitepaper describes the implementation as written in the provided Pine Script and explains the reasoning behind the weight hierarchy, threshold mapping, and event-driven alert design.

2. Problem statement and design goals

Gold is sensitive to multiple cross-asset forces at the same time. Watching all of them manually is cognitively expensive, and naive one-variable heuristics such as 'oil down means gold up' are often incomplete. The script addresses that problem by compressing several related macro signals into a single regime label.

- Transparency: the user should be able to explain every output from the code.
- Interpretability: the indicator should show why a regime is bullish or bearish.
- Real-time usability: signals must update on the chart timeframe without external data wrangling.
- Flexibility: symbol inputs must be editable because TradingView feeds differ by plan and vendor.
- Moderate noise control: alerts should fire on state transitions, not continuously inside a regime.

The design deliberately favors explainability over statistical sophistication. That choice makes the model easy to audit, tune, and extend.

3. System architecture

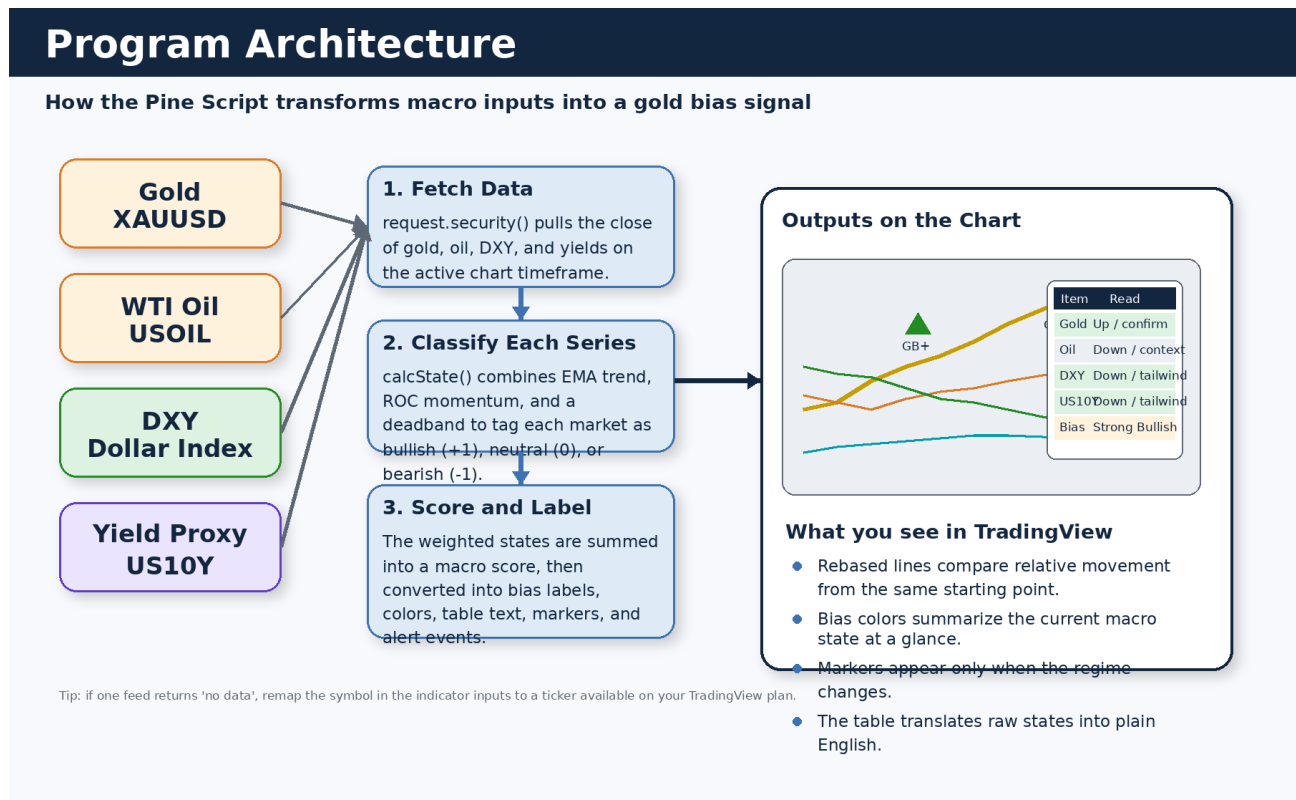


Illustration 1. The program flow from market inputs to chart outputs.

The architecture can be understood as a six-stage pipeline: symbol acquisition, series retrieval, state classification, weighted scoring, regime labeling, and output rendering. Each stage is independent enough to be tested conceptually on its own.

- Symbol acquisition uses `input.symbol()` so the user can swap feeds without editing core logic.
- Series retrieval uses `request.security()` to bring each instrument onto the active chart timeframe.
- Classification converts each raw series into a state in $\{-1, 0, +1\}$.
- Scoring aggregates the states into a macro score with asymmetric weights.
- Regime labeling maps the score to $\{-2, -1, 0, +1, +2\}$.
- Rendering turns the regime into colors, markers, table rows, and alerts.

4. Data layer and state classification

The data layer consists of four close series: gold, oil, DXY, and yields. The script checks data availability through `dataOk` and avoids producing a normal bias if any required series is missing.

```
gold = getSeries(goldSymbol)
oil = getSeries(oilSymbol)
dxy = getSeries(dxySymbol)
yld = getSeries(yldSymbol)

dataOk = not na(gold) and not na(oil) and not na(dxy) and not na(yld)
```

State classification is handled by `calcState()`. The function returns the discrete state together with the EMA and ROC used to compute it. The EMA acts as a directional filter; the ROC and deadband act as a significance filter.

State Engine

How `calcState()` turns a raw price series into -1, 0, or +1

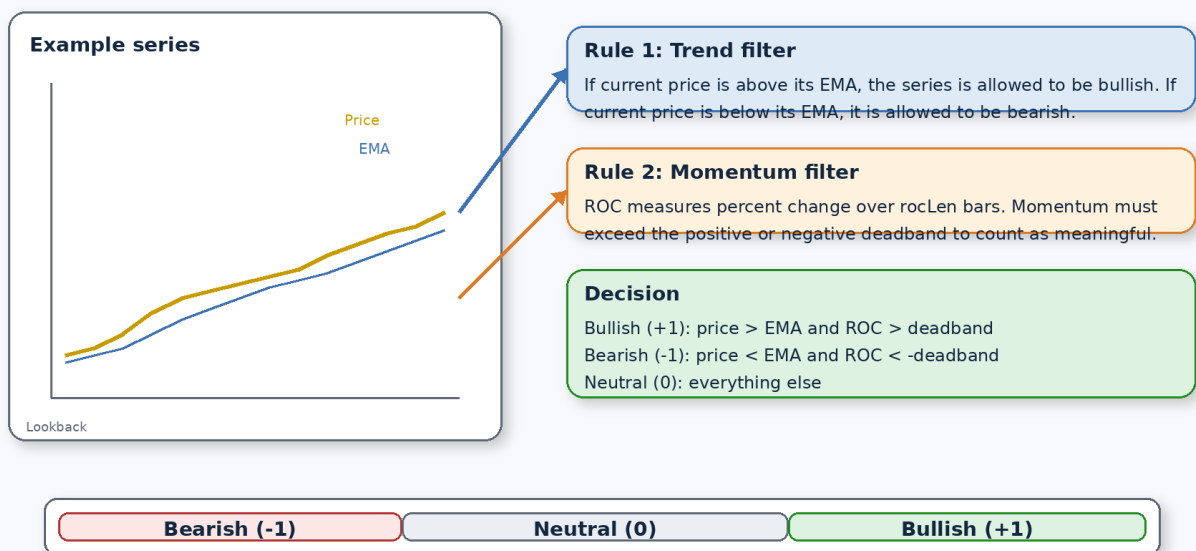


Illustration 2. The state engine as a discrete classifier.

Formally, for a series $s(t)$, the state is bullish when $s(t)$ is above its EMA and the percent change over `rocLen` bars exceeds the deadband. It is bearish when both conditions reverse. All other cases are neutral. This tri-state compression sacrifices granularity, but it gains robustness and interpretability.

5. Scoring and bias mapping

Scoring Model

Why gold, DXY, yields, and oil do not have equal influence

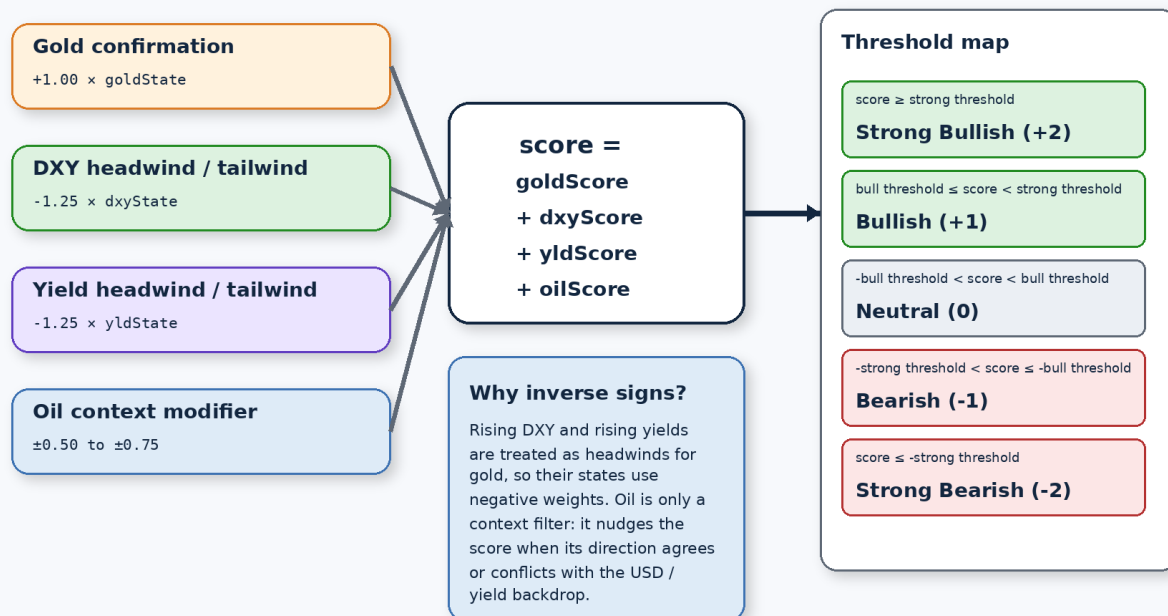


Illustration 3. Weighted score construction and threshold-based bias mapping.

The score is constructed as: $\text{score} = \text{goldScore} + \text{dxyScore} + \text{yldScore} + \text{oilScore}$. Gold contributes a confirmation term with unit weight. DXY and yields contribute inverse terms with larger absolute weights. Oil contributes a smaller context term that is conditional on the direction of the other macro drivers.

Term	Definition	Purpose
goldScore	$1.00 \times \text{goldState}$	Confirms that gold itself is aligned with the backdrop
dxyScore	$-1.25 \times \text{dxyState}$	Captures the inverse dollar effect on gold
yldScore	$-1.25 \times \text{yldState}$	Captures the inverse yield effect on gold
oilScore	Context-dependent ± 0.50 or ± 0.75	Adds inflation/growth context without dominating the signal

The oil term depends on $\text{macroTailwind} = (-\text{dxyState}) + (-\text{yldState})$. That means oil is interpreted through the lens of the dollar and yields rather than in isolation.

Oil state and backdrop	oilScore	Interpretation
Oil down and $\text{macroTailwind} \geq 1$	+0.75	Falling oil aligns with a gold-friendly USD/yield backdrop
Oil down and $\text{macroTailwind} \leq -1$	-0.75	Falling oil reinforces a gold-unfriendly backdrop
Oil up and $\text{macroTailwind} \geq 1$	+0.50	Rising oil adds mild reflation support when the main drivers are already favorable
Oil up and $\text{macroTailwind} \leq -1$	-0.50	Rising oil becomes a headwind when the main drivers are unfavorable

After scoring, the script maps the numeric result into strong bullish, bullish, neutral, bearish, or strong bearish regimes by comparing the score with user-configurable thresholds.

6. Alert semantics and visualization

The output layer is intentionally redundant: the same internal state is exposed through multiple visual and operational channels so the user can consume the signal in the way that fits their workflow best.

Dashboard Anatomy

What each visible element means when the indicator is on a chart

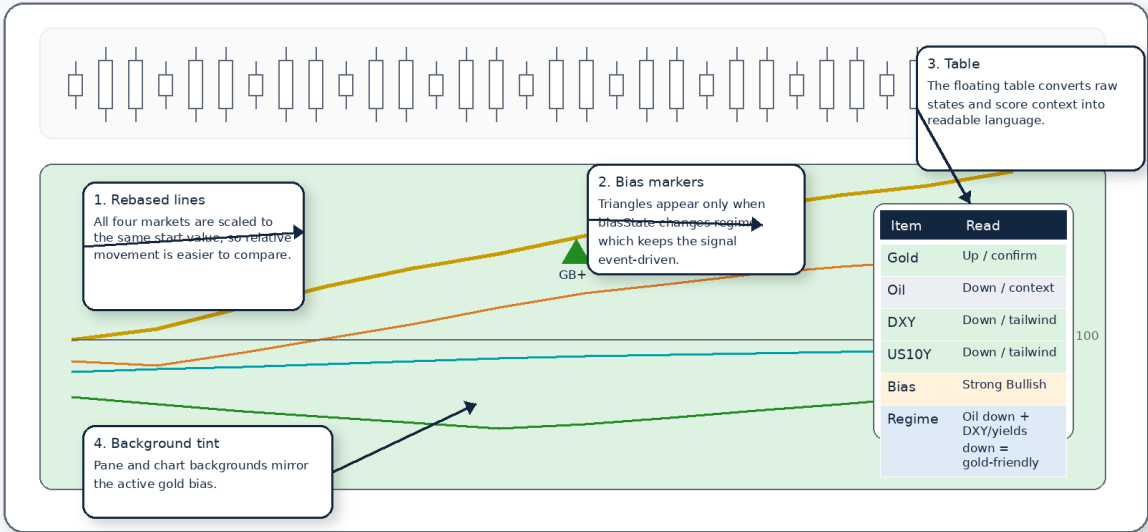
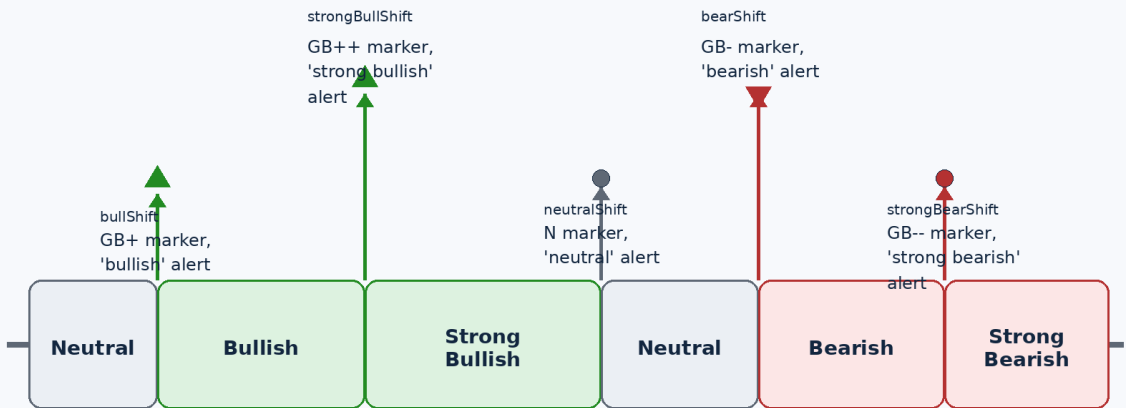


Illustration 4. Visualization surfaces: rebased lines, background tint, markers, and floating table.

Signal Flow

How regime changes become markers and alert conditions



The script does not alert continuously inside a regime. It alerts only when the biasState changes from one regime to another. That keeps notifications event-driven rather than noisy.

Illustration 5. Alert logic is transition-based rather than continuously evaluative.

- Rebased lines normalize all series to a shared starting value for visual comparison.

- Pane and chart backgrounds mirror the current bias color.
- Markers identify transition points such as GB+, GB++, GB-, and GB--.
- The floating table translates states into readable language.
- `alertcondition()` emits event-driven notifications when the regime changes.

This layered presentation is useful because different users consume information differently. A discretionary trader may care most about color and table text, while an automation-oriented user may care most about alert transitions.

7. Design rationale

The relative weights encode the model's belief hierarchy. DXY and yields carry the largest absolute influence because they are treated as the main macro headwind or tailwind variables for gold. Gold itself receives a slightly smaller weight because it serves as confirmation rather than as an external driver. Oil receives the smallest contribution because its effect is interpreted as contextual.

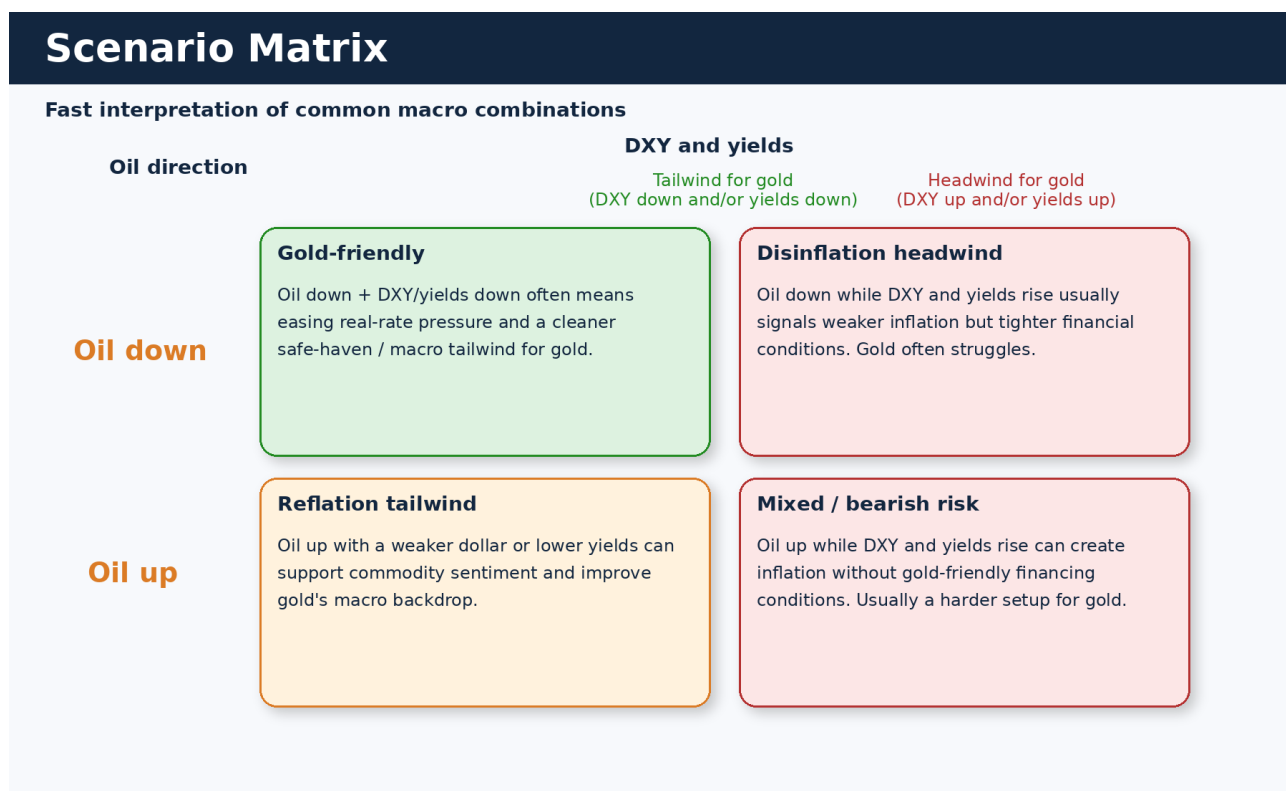


Illustration 6. Scenario matrix: the same oil move can imply very different things depending on DXY and yields.

- Discrete states are easier to reason about than raw z-scores in a TradingView overlay indicator.
- The deadband reduces churning around zero momentum.
- Transition-based alerts reduce operator fatigue.
- Editable symbols solve practical feed-availability problems without touching the logic.

The result is not a universal truth about gold. It is a practical operational model: transparent enough to audit, but structured enough to support repeatable decision-making.

8. Limitations and failure modes

The model is intentionally simple, and that simplicity creates limitations. Users should understand those limitations before treating the signal as authoritative.

- Static weights: the model does not re-estimate weights by regime, volatility, or macro cycle.
- Proxy yields: nominal yields are used instead of a direct real-yield measure.
- Single-timeframe sampling: all inputs are pulled to the chart timeframe, which can hide higher-timeframe disagreement.
- No volatility normalization: the state engine treats the deadband as fixed rather than adapting to market volatility.
- Feed dependency: different data vendors can produce small differences in timing or price levels.
- No empirical performance claims: the script as provided is a rules engine, not a validated research report.

Failure modes often appear around major macro events, cross-asset decouplings, or periods when gold trades more on idiosyncratic flows than on the standard USD-and-yield framework.

9. Validation framework

No historical validation is embedded in the current script. A disciplined next step is to measure whether each bias state actually aligns with future gold behavior. The validation framework below is proposed rather than already completed.

- Forward-return study: measure 1-bar, 5-bar, and 10-bar gold returns after each bias state.
- Persistence study: measure how long each regime tends to last once entered.
- Transition study: compare performance after fresh regime shifts versus already-established regimes.
- Sensitivity study: grid-search EMA, ROC, deadband, and threshold settings.
- Robustness study: repeat the analysis across multiple gold feeds and timeframes.

The most useful outcome is not just average return by state, but the interaction between return, variance, and regime persistence. A state that is only slightly predictive but highly persistent can still be valuable as a trade filter.

10. Extension roadmap

The current indicator is a strong base for future work. Several extensions are natural next steps.

- Strategy version with entries, exits, ATR stops, and position sizing.
- Real-yield proxy or TIPS-based input instead of nominal yields alone.
- Multi-timeframe confirmation so the chart timeframe must agree with a higher timeframe.
- Volatility-aware deadband that expands and contracts with market conditions.
- Correlation or regime filter to disable the model when macro relationships weaken.
- Output export or webhook automation tied to alert transitions.

These upgrades would increase sophistication, but they should preserve the current script's best feature: transparency.

Appendix A. Parameter reference

Parameter	Default	Function	What changes when you raise it
emaLen	34	Trend filter length	Trend state becomes slower and cleaner
rocLen	5	Momentum lookback	Momentum reacts more slowly
deadbandPct	0.20	Momentum significance filter	More moves are classified as neutral
rebaseLen	100	Visual comparison anchor	Comparison window becomes longer
bullThreshInput	1.50	Bullish threshold	Fewer bullish labels
strongThreshInput	3.00	Strong threshold	Fewer strong bullish/bearish labels
showPaneBg	true	Pane tint	Display preference only
showChartBg	true	Chart tint	Display preference only
showMarkers	true	Transition markers	Display preference only
colorBars	false	Bar coloring	Display preference only

Appendix B. Pseudocode

```
load gold, oil, DXY, and yield series
if any series is missing:
  show 'Check symbols' state
else:
  classify each series as -1, 0, or +1 using EMA + ROC + deadband
  compute goldScore, dxyScore, yldScore
  compute macroTailwind from DXY and yields
  compute oilScore from oilState and macroTailwind
  sum all terms into score
  map score to strong bearish / bearish / neutral / bullish / strong bullish
  color the pane and chart
  plot rebased lines
  update the dashboard table
  emit alerts only when the regime changes
```

This pseudocode omits rendering details, but it preserves the full logical sequence of the model.

Educational material only. Not investment advice.